DRAMA: Next-Gen Dynamic Orchestration for Resilient Multi-Agent Ecosystems in Flux

Anonymous CVPR submission

Abstract

Embodied Multi-Agent Systems (EMAS) have proven highly effective in addressing complex tasks through coordinated collaboration among heterogeneous agents. However, real-world environments and task specifications are inherently dynamic, exhibiting frequent changes, uncertainty, and variability. Despite these characteristics, most existing EMAS frameworks employ static architectures with fixed agent capabilities and rigid task allocation strategies, which substantially constrain their adaptability to evolving conditions. This inflexibility presents significant challenges to maintaining robust and efficient multi-agent cooperation in dynamic and unpredictable settings.

To address these limitations, we propose ***DRAMA****—DRAMA: Next-Gen* ***D****ynamic O****R****chestr****A****tion for Resilient* ***M****ulti-****A****gent Ecosystems tailored for rapidly changing environments. DRAMA adopts a multilayer architecture that incorporates three principal mechanisms: (*i*) adaptive scheduling via an affinity-driven mechanism, (*ii*) fault-tolerant continuity through hierarchical trust-chain task takeover, and (*iii*) collective spatial intelligence that consolidates distributed observations for predictive reasoning. Together, these components enable event-triggered rescheduling and decentralized fault recovery, ensuring uninterrupted task execution amid agent arrivals, dropouts, or recoveries. Extensive experiments in the embodied VirtualHome-Social environment demonstrate that DRAMA achieves a 7% improvement in runtime efficiency, and a 10% increase in throughput compared with state-of-the-art baselines, while maintaining superior stability and robustness under dynamic agent populations.*

1. Introduction

The rapid advancement of Large Language Models (LLMs) has significantly broadened their capabilities across diverse domains, including natural language processing [1, 22, 32] and code generation [6, 9, 11, 19, 38]. These advancements have concurrently accelerated progress in multi-agent systems (MAS), endowing agents with enhanced planning, reasoning, and autonomous collaboration abilities applicable to a variety of scenarios [12, 15, 30]. LLM-based multi-agent systems (LLM-MAS) have demonstrated outstanding performance in collaborative planning and emergent problem-solving across diverse tasks [5, 14, 28]. When integrated into physical or simulated environments, such intelligent agents become embodied, enabling them to address long-horizon, open-world tasks that increasingly mirror human-like behavioral patterns [2, 17, 25, 29].

Despite these advances, most existing Embodied Multi-Agent System (EMAS) frameworks are built upon static architectures in which agent capabilities and task allocations remain fixed. This rigidity severely constrains adaptability and robustness in dynamic, real-world environments. In practice, both team composition and task conditions are inherently fluid—agents may join, depart, or fail, and task specifications can evolve unpredictably [8, 37]. Such volatility underscores the necessity of EMAS frameworks that support adaptive and resilient task allocation, thereby maintaining effective collaboration and stable performance amid continual changes in team structure and task demands. Addressing these challenges is crucial for the practical deployment of MAS in complex and evolving environments.

To address these limitations, we propose **DRAMA**—Next-Gen **D**ynamic O**R**chestr**A**tion for Resilient **M**ulti-**A**gent Ecosystems in Flux. DRAMA is built upon a multilayer architecture that integrates adaptive strategic layer, collective intelligence layer, and autonomous layer. In the **Strategic Layer**, an affinity matrix models real-time agent–task compatibility and is optimized using the Hungarian algorithm to achieve globally coherent task allocation. A hierarchical **trust-chain** mechanism ensures fault-tolerant continuity, where each task is protected by an ordered set of backup agents capable of immediate takeover in case of failure. The **Collective Intelligence Layer** aggregates spatial observations from distributed agents to learn probabilistic object–location priors, thereby enhancing perception and planning efficiency.

Finally, the **Autonomous Layer** equips each agent with dual roles as executor and guardian, enabling real-time perception, action execution, and peer monitoring.

We evaluate DRAMA extensively in the embodied environment VirtualHome-Social [26], benchmarking it against representative EMAS baselines. Results demonstrate that DRAMA is the only framework capable of reliably handling both agent dropout and recovery scenarios. Moreover, it achieves a 17% improvement in runtime efficiency, a 13% reduction in task conflict, and a 10% gain in throughput compared to existing methods, while maintaining superior robustness and adaptability under frequent agent turnover and dynamic task conditions.

The main contributions are summarized as follows:

- We propose **DRAMA**, the Next-Gen **D**ynamic **O**R**chestr**A**tion for Resilient **M**ulti-**A**gent Ecosystems in Flux that supports real-time adaptation to agent arrivals, dropouts, and recoveries through event-triggered task reallocation and flexible resource management.
- We introduce a unified abstraction that models both agents and tasks as resource entities, enabling affinity-driven scheduling via the Hungarian algorithm and promoting modular, decoupled coordination in heterogeneous multi-agent environments.
- We design a three-layer architecture that integrates adaptive scheduling, collective intelligence, and fault-tolerant execution through trust-chain–based task takeover and decentralized collaboration.
- We conduct extensive experiments in the embodied VirtualHome-Social environment, demonstrating that DRAMA consistently outperforms state-of-the-art baselines in efficiency, resource utilization, and robustness.

2. Background

This section reviews the fundamental concepts of current EMAS, highlights current challenges, and outlines the motivation behind our research.

2.1. Embodied Multi-Agent Systems

An EMAS consists of a collection of autonomous agents $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$ that cooperatively or competitively perform a set of tasks $\mathcal{Q} = \{\mathtt{q}_1, \mathtt{q}_2, \ldots, \mathtt{q}_M\}$ in a shared environment $\mathcal{E}$. Each embodied agent a_i functions as an autonomous control loop interacting with the environment and can be formally represented as

$$a_i = \langle S_i, O_i, A_i, \pi_i \rangle, \tag{1}$$

where S_i denotes its local state space, O_i its observation space, A_i its available action set, and π_i its policy for decision-making and control. The allocation of agents to tasks at time t can be described by a mapping $f_t : \mathcal{Q} \to \mathcal{A}$, where $f_t(\mathtt{q}_j)$ specifies the agent responsible for task $\mathtt{q}_j$. The global system objective over time horizon T is to maximize a collective utility function:

$$\max_{\{f_t\}_{t=1}^{T}, \{\pi_i\}_{i=1}^{N}} U(\{f_t\}_{t=1}^{T}, \mathcal{A}, \mathcal{Q}, \mathcal{E}, T), \tag{2}$$

where $U(\cdot)$ evaluates the overall task performance.

2.2. Motivation

Although the general formulation of EMAS supports dynamic agent-task assignments over time, most existing frameworks adopt a static assignment paradigm in practice. Specifically, the assignment function is determined at initialization ($t = 0$) and remains fixed throughout the system's operation, i.e.,

$$f_t = f_0, \quad \forall t \in [1, T]. \tag{3}$$

The system objective thus reduces to:

$$\max_{f_0, \{\pi_i\}_{i=1}^{N}} U(f_0, \mathcal{A}, \mathcal{Q}, \mathcal{E}, T) \tag{4}$$

While the static assignment approach simplifies system design and deployment, it introduces several critical limitations in dynamic, real-world environments:

- **Agent and Task Dynamism:** Agents may join, leave, or change capabilities over time; tasks may arrive, evolve, or be reprioritized. Static assignments prevent the system from adapting to these changes.
- **Suboptimal Resource Utilization:** Fixed assignments can result in idle agents, overloaded tasks, or inefficient execution, particularly as environmental conditions shift.
- **Lack of Real-Time Responsiveness:** Without mechanisms to monitor and adapt f_t, the system cannot respond effectively to failures, delays, or unforeseen events.
- **Limited Scalability and Robustness:** The inability to reallocate resources dynamically constrains system performance and resilience in large-scale, open, or uncertain settings.

These challenges underscore the necessity for EMAS architectures that enable adaptive, real-time agent-task assignment, leveraging continuous monitoring and feedback to optimize system utility in dynamic environments.

3. Methodology

In this section, we present **DRAMA**, a modular **three-layer framework** that unifies agents and tasks as resource entities for adaptive and resilient multi-agent orchestration. The architecture comprises: (i) a *Strategic Layer* for affinity-based task allocation and fault-tolerant scheduling; (ii) a *Collective Intelligence Layer* that aggregates distributed perceptions for shared spatial reasoning; and (iii) an *Autonomous Layer* enabling agents to execute tasks and monitor peers. Together, these layers support real-time adaptation and continuous task execution under dynamic team compositions and environments. An overview is illustrated in Figure 1.

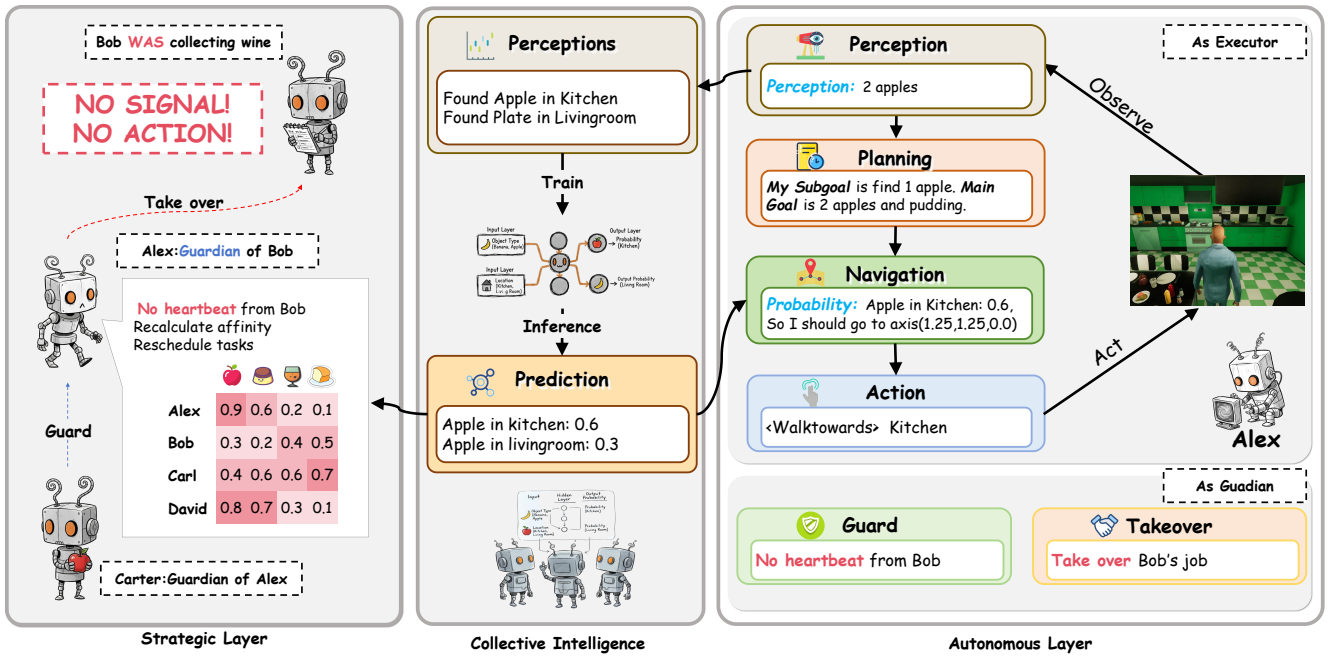

Figure 1. **Overview of the DRAMA framework**. It adopts a three-layer architecture—Strategic, Collective Intelligence, and Autonomous layers—for adaptive scheduling, collective spatial reasoning, and resilient task execution under dynamic agent arrivals and dropouts.

3.1. Strategic Layer

Building upon the multi-agent formulation introduced in Section 2, the **strategic layer** governs the adaptive optimization of the agent–task mapping f_t and reinforces robustness through decentralized fault tolerance. As depicted in Figure 2, this layer integrates three components: (1) an affinity matrix capturing agent–task compatibility, (2) task scheduling via the Dual-Capacity Hungarian Assignment algorithm, and (3) trust-chain construction that provides hierarchical recovery for resilient task execution.

Affinity Computation. At time step t, the instantaneous affinity between agent a_i and task q_j is formulated as a weighted combination of the agent's available workload capacity and the predicted spatial distance between them:

$$\mathcal{S}_{ij,t} = w_1 \cdot v_i(t) - w_2 \cdot \mathrm{Dist}\big(\mathbf{x}_i(t), \hat{\mathbf{p}}_j(t)\big), \qquad (5)$$

where $v_i(t)$ represents the current availability of agent a_i, $\mathbf{x}_i(t)$ denotes its current position, and $\hat{\mathbf{p}}_j(t)$ indicates the predicted location of task q_j, which is estimated by the collective intelligence layer based on the spatial distribution $P(o_j, p)$. $\mathrm{Dist}(\cdot)$ refers to the Euclidean distance, and $w_1, w_2 > 0$ are weighting coefficients (see Section 3.2).

Task Scheduling To achieve globally coherent task allocation, the system searches for the assignment mapping that

Algorithm 1 Dual-Capacity Hungarian Assignment

Require: Agents $A = \{a_i\}_{i=1}^{N}$, Tasks $T = \{t_j\}_{j=1}^{M}$, Affinity function Aff
Ensure: Mapping $\mathcal{M} : A \to \mathcal{P}(T)$ where $|\mathcal{M}(a_i)| \le 2$

```
V ← ∅
for each agent a_i ∈ A do
    V ← V ∪ {v_{i,1}, v_{i,2}}        ▷ Each agent provides two slots
end for
1. Create virtual agents and cost matrix
C ← new |V| × |T| cost matrix
for each virtual slot v_{i,k} ∈ V do
    for each task t_j ∈ T do
        C[v_{i,k}, t_j] ← −Aff(a_i, t_j)
    end for
end for
2. Solve as a standard assignment problem
K ← max(|V|, |T|)
C' ← new K × K matrix, filled with ∞
C'_{1..|V|,1..|T|} ← C
P ← HungarianAlg(C')              ▷ P is a set of optimal pairs
3. Consolidate results back to real agents
M(a_i) ← ∅ for all a_i ∈ A
for each pair (v_{i,k}, t_j) ∈ P do
    if t_j ∈ T then                ▷ Filter dummy tasks
        M(a_i) ← M(a_i) ∪ {t_j}
    end if
end for
return M
```

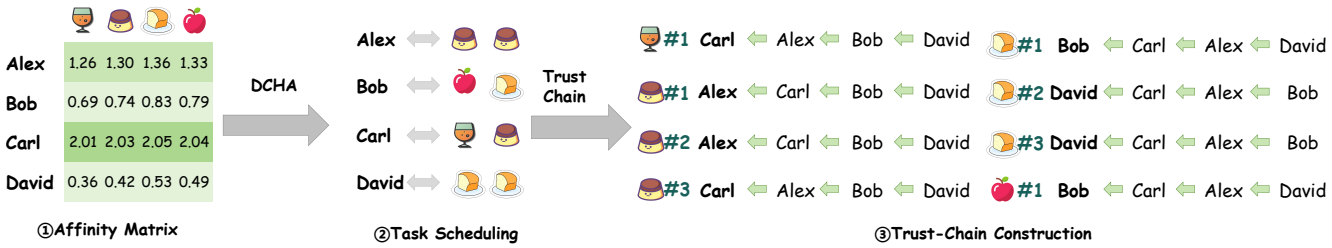

Figure 2. **A demonstration of the task scheduling and trust-chain mechanism in the Strategic Layer.** The left matrix shows dynamic affinity scores between agents and tasks, derived from historical cooperation and current context. The DCHA then produces optimal one-to-many task assignments (middle). Each assigned agent becomes the head of a hierarchical *trust chain* (right), where subsequent agents act as guardians to monitor and take over if predecessors fail. This design enables adaptive and fault-tolerant multi-agent scheduling.

maximizes the total affinity over all tasks:

$$f_t^* = \arg\max_{f_t} \sum_{j=1}^{M} \mathcal{S}_{f_t(q_j)j,t}, \quad \text{s.t. } f_t : \mathcal{Q} \to \mathcal{A} \text{ (one-to-one)}. \tag{6}$$

The optimal mapping f_t^* is determined by our proposed Dual-Capacity Hungarian Assignment (DCHA) algorithm. This method adapts the classic Hungarian algorithm [24] for our one-to-many assignment problem, ensuring a globally optimal allocation that maximizes the total context-aware suitability. The full procedure is detailed in Algorithm 1.

Trust-Chain Construction. After the primary task assignment is obtained from the DCHA algorithm, the system constructs a hierarchical trust chain to provide fault-tolerant continuity for each task. For task q_j, let $\mathcal{S}_{:,j,t}$ denote the affinity column corresponding to all agents. Agents are ranked in descending order of affinity:

$$a_{(1)} \succ a_{(2)} \succ a_{(3)} \succ \cdots \succ a_{(N)}.$$

The agent selected by the DCHA algorithm, $a_{(1)} = f_t^*(q_j)$, serves as the *primary executor* and occupies the head of the trust chain. Subsequent agents are organized into an ordered guardian sequence:

$$a_{(1)} \to a_{(2)} \to a_{(3)} \to \cdots \to a_{(N)},$$

where the arrow denotes the guarding direction. Each guardian supervises the operational state of its predecessor and remains on standby for takeover.

If the executor $a_{(1)}$ fails, responsibility is immediately transferred to its guardian $a_{(2)}$; if $a_{(2)}$ fails subsequently, $a_{(3)}$ becomes active, and so forth. This design ensures that the executor is always positioned as the foremost node in its guardian chain—the entity being protected— and that every task possesses an ordered backup sequence for recovery. After each takeover event, local recalibration updates workload balance and affinity relationships to maintain coherent system scheduling.

3.2. Collective Intelligence Layer

The **collective intelligence layer** functions as the shared spatial cognition component of the system. It integrates distributed observations from multiple agents, learns spatial regularities of object occurrences across environments, and provides predictive spatial priors of task or object locations to support the strategic layer.

Collective Experience Integration Each agent a_i initially explores the environment to accumulate spatial experience through *visual observations*, represented as $\mathcal{H}_{i,t} = \{(o_k, p_k) \mid k < t\}$, where o_k denotes a visually perceived object or task type, and p_k indicates its corresponding spatial coordinates (e.g., room or location). After several exploration or task execution, all agents exchange and integrate their visual observation histories as $\mathcal{H}_t = \bigcup_{i=1}^{N} \mathcal{H}_{i,t}$. This shared dataset constitutes a collective spatial memory that captures the co-occurrence relationships between objects and locations, serving as the empirical foundation for estimating spatial distributions $\mathcal{P}(o, p)$.

Collective Spatial Inference Based on the aggregated observation history $\mathcal{H}_t$, the collective intelligence layer learns the spatial likelihood of objects or tasks across the environment. Rather than relying solely on empirical frequency estimation, which provides only a coarse prior, we train an MLP-based predictor $f_\theta(\cdot)$ to approximate the joint distribution $\mathcal{P}(o, p)$ capturing higher-order spatial correlations between contextual factors and object locations. Given contextual input x_t (e.g., current room features, semantic cues, and co-occurring objects), the network outputs the probability of an object o appearing at position p:

$$\hat{\mathcal{P}}_\theta(o, p \mid x_t) = f_\theta(x_t).$$

The predicted most probable location of a queried object or task o_j is then obtained as $\hat{p}_j = \mathbb{E}_{p \sim \hat{\mathcal{P}}_\theta(o_j, p)}[p]$ or $\arg\max_p \hat{\mathcal{P}}_\theta(o_j, p)$. This design enables generalization beyond explicitly observed samples and supports inference in unexplored areas. See details in the supplementary.

3.3. Autonomous Layer

The **autonomous layer** constitutes the self-regulated control architecture of each agent. It integrates perceptual processing, goal planning, motion execution, and fault monitoring to achieve adaptive and resilient autonomy. Every agent functions simultaneously as an *executor*, responsible for perception and action, and as a *guardian*, responsible for peer supervision and fault handling.

Executor Role In the executor pathway, the autonomous layer establishes a continuous perception–action loop that grounds high-level goals in concrete environmental interactions. Each agent maintains a *temporal context* by systematically recording its past actions, observations, communications, and the evolving state of its surroundings. This persistent memory allows agents to adapt their behavior based on accumulated experience. To manage increasing task complexity and the expanding history of observations, agents employ a *hierarchical memory management* strategy: information related to ongoing primary and critical subtasks is retained in detail to support timely decision-making, recent but less crucial events are compressed into summarized representations, and outdated or irrelevant data are selectively discarded to conserve computational resources.

Upon receiving a coarse-grained task from the control plane, the agent decomposes the high-level objective into finer-grained, actionable subtasks tailored to the current environment. This process leverages real-time perceptual inputs—such as spatial layout, object locations, and environmental changes—together with memory of past actions and outcomes. The *perception module* transforms raw sensory information into structured scene representations for reasoning, the *planning module* formulates executable subgoals, and the *navigation module* integrates predicted object or task locations $\hat{p}_j$ from the collective intelligence layer to generate trajectory plans and motion commands. Finally, the *action module* executes these plans through motor control, directly interacting with the environment.

Guardian Role Running parallel to the executor process, the guardian mechanism safeguards operational robustness and continuity. The *guard module* continuously monitors heartbeat signals and status indicators from peer agents to detect anomalies such as latency, dropouts, or malfunction. When such irregularities are identified, the *takeover module* initiates a decentralized recovery procedure: it evaluates the pending responsibilities of the affected agent, reallocates unfinished tasks to suitable peers based on current workload and spatial proximity, and triggers re-execution through their executor pathways. This trust-chain–driven reallocation ensures seamless task recovery, maintains system integrity, and preserves global performance under agent failures or dynamic team composition.

4. Experiment Setup

4.1. Dataset

To comprehensively evaluate the effectiveness and robustness of various embodied multi-agent coordination methods, we build upon the VirtualHome-Social environment [26, 27, 35], a Unity-based simulation platform. We observe that the default task configurations in C-WAH [27] are relatively simple and may not adequately distinguish the capabilities of advanced approaches. To address this limitation, we design a more challenging experimental setting by increasing both the number of objects and the diversity of task types, thereby introducing greater complexity and requiring more sophisticated cooperation strategies. All trials are repeated in the event of simulator-induced failures. Details of the dataset are provided in the supplementary.

4.2. Baselines

To comprehensively evaluate our framework, we compare it against several representative baselines, each reflecting a distinct coordination or planning paradigm. Since these baselines were not originally designed for VirtualHome-Social environment, we adapted and re-implemented them for this setting. For robustness evaluation, we adhere to each baseline's original design, including their lack of support for task handover where applicable. For efficiency evaluation, however, we equip all methods with additional capabilities to ensure comparably high success rates: specifically, each method is provided with both the final target and current progress information as input. These enhancements enable a fair comparison of the efficiency of different task assignment strategies across baselines.

The compared methods are:

- **CoELA** [35]: Enables agents to plan, communicate, and cooperate via inter-agent messaging and teamwork.
- **MCTS** [10]: Employs Monte Carlo Tree Search for planning-based task allocation, selecting high-reward solutions within a fixed search budget.
- **AgentVerse-Static (AV-Static)** [4]: Assigns agent roles and tasks only at initialization; assignments remain fixed.
- **AgentVerse-Dynamic (AV-Dynamic)**: Triggers reallocation among agents only upon task completion, periodically redistributing remaining tasks.
- **ProAgent** [33]: Adopts a fully decentralized approach—agents coordinate via communication, prediction, and negotiation, without a central scheduler.

4.3. Metrics

To rigorously assess multi-agent cooperation in dynamic environments, we employ a comprehensive set of metrics that evaluate both outcome and process-level performance.

Table 1. **Effectiveness comparison across different scenarios and methods.** Each method is evaluated with three metrics: Average Steps (AS), Conflict Rate (CR), and Throughput (TP).

Method	Metrics	Static			Dropout		Addition		Recovery	Replacement
		3	4	5	4→3	5→4→3	4→5	3→4→5	4→3→4	4→3→4′
Comparison Methods										
CoELA	AS	70.511	54.956	52.293	61.841	57.556	54.415	57.565	60.791	60.977
	CR	0.0614	0.0883	0.173	0.065	0.102	0.121	0.100	0.085	0.090
	TP	0.169	0.222	0.240	0.182	0.195	0.228	0.198	0.189	0.182
AV-dynamic	AS	72.784	56.296	47.974	59.297	56.244	54.732	61.342	59.000	61.422
	CR	0.060	0.074	0.128	0.048	0.089	0.100	0.076	0.062	0.068
	TP	0.151	0.203	0.241	0.193	0.204	0.234	0.222	0.198	0.193
AV-static	AS	68.447	52.727	49.450	60.568	55.024	51.539	56.846	56.822	59.781
	CR	0.071	0.095	0.122	0.058	0.108	0.106	0.084	0.064	0.073
	TP	0.160	0.212	0.240	0.195	0.207	0.230	0.221	0.207	0.190
ProAgent	AS	64.861	51.560	47.683	59.761	54.955	48.717	54.804	55.296	56.711
	CR	0.038	0.064	0.109	0.065	0.111	0.136	0.092	0.080	0.087
	TP	0.166	0.211	0.237	0.196	0.208	0.233	0.205	0.214	0.197
MCTS	AS	92.200	77.330	67.260	88.780	86.850	72.000	80.160	83.330	82.070
	CR	0.033	0.023	0.044	0.035	0.026	0.019	0.014	0.028	0.029
	TP	0.133	0.158	0.174	0.138	0.105	0.151	0.111	0.113	0.114
Our Method										
DRAMA	AS	**59.977**	**50.242**	**46.697**	**57.205**	**52.796**	**48.213**	**53.367**	**55.069**	**55.429**
	CR	**0.027**	**0.062**	**0.083**	**0.026**	**0.076**	**0.068**	**0.057**	**0.044**	**0.061**
	TP	**0.189**	**0.217**	**0.252**	**0.200**	**0.216**	**0.235**	**0.211**	**0.204**	**0.205**

These metrics capture not only task completion but also system efficiency, adaptability, and resource utilization.

Average Steps (AS): Quantifies temporal efficiency by calculating the mean number of steps taken by the agent completing the final task in each episode, indicating overall latency and the system's *effectiveness* in minimizing completion time for complex tasks.

Conflict Rate (CR): Evaluates the degree of coordination by measuring the proportion of steps in which two or more agents simultaneously execute the same action on the same object. A higher conflict rate indicates redundant or conflicting behaviors, while a lower rate reflects stronger cooperative synchronization.

Throughput (TP): Measures task-level productivity by computing the ratio between the number of successfully completed target objects and the total number of steps. It reflects both speed and consistency in task execution.

4.4. Scenario Design

To thoroughly evaluate the robustness and adaptability of our framework, we design a set of experimental scenarios that cover both stable and dynamic agent populations. In our implementation, the maximum number of embodied agents is limited to five, representing the upper bound of the current system capacity.

- **Static Scenarios:** The agent population remains fixed throughout the experiment. We consider three static configurations consisting of 3, 4, and 5 agents, which respectively represent small, medium, and full-capacity settings.
- **Dynamic Scenarios:** To evaluate the framework's adaptability, we designed scenarios simulating agent failures, resource scaling, and team turnover.
  The dynamic scenarios are categorized as follows:
  - *Agent Dropout:* Agents are removed with patterns of $4 \rightarrow 3$ and cascading $5 \rightarrow 4 \rightarrow 3$.
  - *Agent Addition:* New agents are introduced with patterns of $4 \rightarrow 5$ and $3 \rightarrow 4 \rightarrow 5$.
  - *Agent Turnover:* We test two complex turnover events:
    - *Recovery:* An agent temporarily leaves and later rejoins at same location($4 \rightarrow 3 \rightarrow 4$).
    - *Replacement:* An agent leaves and is replaced by a new agent at a different location ($4 \rightarrow 3 \rightarrow 4'$).

To ensure fair comparison across experiments, the timing of agent removal or addition is aligned to fixed progress ratios of the overall task execution, so that all methods experience structural changes at equivalent temporal stages. These scenarios collectively evaluate the framework's ability to maintain stability, adapt to resource fluctuations, and recover from agent interruptions under varying population scales.

Table 2. Scenario success comparison across different environmental conditions. ✓ indicates success and ✗ indicates failure. **DRAMA** successfully supports all scenario types.

Scenario	CoELA	MCTS	ProAgent	AV-static	AV-dynamic	DRAMA
Static	✓	✓	✓	✓	✓	✓
Dropout	✗	✗	✗	✗	✗	✓
Addition	✓	✓	✓	✓	✓	✓
Recovery	✗	✗	✗	✗	✗	✓

5. Experimental Results

5.1. Effectiveness of DRAMA

Efficiency. To evaluate the efficiency of **DRAMA**, we compare the Average Steps (AS) across all methods, as presented in Table 1. In static settings, **DRAMA** consistently achieves the lowest AS, indicating clear improvements in execution efficiency. Under dynamic conditions—including agent dropout, addition, recovery, and replacement—**DRAMA** maintains robust superiority, reducing AS by approximately 4% on average and achieving up to 7–8% improvement in highly dynamic scenarios. These results demonstrate that **DRAMA** substantially enhances operational efficiency and responsiveness in evolving multi-agent environments.

Conflict Rate. Although the rule-based MCTS framework achieves slightly lower CR due to its static and predefined scheduling policy, it cannot adapt to real-time agent or task variations. Among all *dynamic scheduling* frameworks, **DRAMA** consistently attains the lowest CR across both static and dynamic settings, demonstrating superior coordination and reduced inter-agent conflicts.

Throughput. Beyond efficiency and coordination, **DRAMA** also achieves the highest throughput across all scenarios, with an overall **10–20%** improvement over existing methods. This gain indicates that DRAMA not only accelerates task completion but also sustains high productivity and consistent performance under fluctuating agent populations and dynamic task conditions.

5.2. Robustness of DRAMA

To comprehensively illustrate framework-level robustness, Table 2 summarizes whether each method successfully completes the task across a variety of environmental scenarios, including static settings and different types of agent dynamics. Specifically, the dropout scenarios simulate reductions in the number of agents from 5 to 3, while the addition scenario involves an increase from 3 to 5 agents. Notably, DRAMA is the only framework that successfully addresses all scenarios. In contrast, all baseline methods fail under agent dropout conditions, as they lack the capacity to reassign incomplete tasks when agents leave.

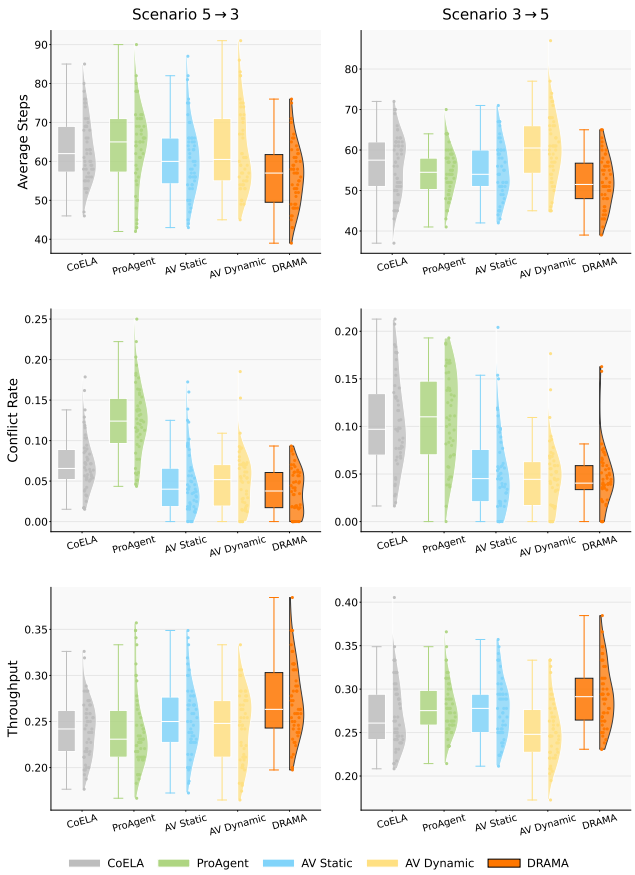

Figure 3. Distributions of Average Steps, Conflict Rate and Throughput across 50 runs for each framework on the same task.

5.3. Stableness of DRAMA

To assess the stability of the framework under challenging conditions, we selected a complex task involving a large number of required objects and conducted 50 independent runs under two dynamic scenarios: agent dropout (5→3) and agent addition (3→5). As illustrated in Figure 3, DRAMA consistently outperforms all baselines, exhibiting lower medians and tighter distributions for *AS* and *CR*, along with higher and more stable *TP* values. These results demonstrate DRAMA's ability to sustain efficient and well-coordinated performance amid agent population fluctuations, underscoring its superior robustness and adaptability in dynamic multi-agent environments.

5.4. Generalization of DRAMA

To evaluate the model-agnostic generalization of DRAMA, we tested it with four diverse foundation models—GPT-4.1, GPT-4o-mini, Qwen-Max, and DeepSeek-V3.2—spanning different architectures and training paradigms. DRAMA

Table 3. Performance of the DRAMA framework when integrated with different backbone foundation models.

Metrics	GPT-4.1	Qwen-max	Deepseek-v3.2	4o-mini
Dynamic: 5 Agents to 3 Agents				
AS	53.422	55.911	53.583	52.796
CR	0.076	0.077	0.045	0.076
TP	0.216	0.208	0.217	0.216
Dynamic: 3 Agents to 5 Agents				
AS	54.022	54.783	51.041	53.377
CR	0.029	0.079	0.036	0.059
TP	0.207	0.208	0.228	0.211

Table 4. **Summary of Ablation Study.** Key results comparing the full model and its ablated variants. "C.I. Layer" denotes the *Collective Intelligence Layer*.

Method	Metrics	Static 3	Static 4	Dropout 4→3
DRAMA	AS	59.977	50.242	57.205
	CR	0.027	0.062	0.026
	TP	0.189	0.217	0.200
w/o C. I. Layer	AS	65.432	56.674	61.021
	CR	0.037	0.054	0.029
	TP	0.171	0.205	0.188
w/o Trust Chain	AS	61.644	53.367	60.851
	CR	0.022	0.046	0.044
	TP	0.182	0.214	0.189

was paired with each model and assessed under two challenging dynamic scenarios: agent dropout (5→3) and agent addition (3→5). As shown in Table 3, the results are highly consistent across all metrics (AS, CR, TP). These findings demonstrate that DRAMA sustains stable performance and efficiency regardless of the underlying model.

5.5. Ablation Study

To further understand the contribution of each core component in DRAMA, we conduct an ablation study focusing on two key mechanisms: the Collective Intelligence Layer and the Trust Chain mechanism. The corresponding quantitative results are summarized in Table 4, while the complete evaluation across all scenarios is reported in the supplementary. The removal of the *C.I. Layer* notably increases *AS* and decreases *TP*, indicating that collective spatial reasoning is essential for enhancing efficiency and task throughput. In contrast, eliminating the *Trust Chain* primarily raises *CR* under dynamic (4→3) conditions, highlighting its pivotal role in preserving coordination and fault tolerance. Overall, these results demonstrate that DRAMA's efficiency largely arises from the *C.I. Layer*, whereas its robustness is chiefly maintained by the *Trust Chain* mechanism.

6. Related Work

Embodied Multi-Agent System Multi-Agent Systems (MAS) refer to computational frameworks comprising multiple autonomous agents that interact, cooperate, or compete to achieve individual or collective objectives [31]. Traditionally, MAS research has focused on developing agents capable of perceiving their environments, making decisions, and coordinating actions to address complex problems beyond the capacity of a single agent. The rapid advancement of LLMs has recently driven a paradigm shift in MAS research. LLMs now serve as powerful and adaptable backbones for agent systems [12, 20], providing strong capabilities in reasoning, planning, and communication. Recent LLM-based MAS frameworks have achieved notable success in collaborative problem-solving [5, 14, 28, 36]. The success of LLM-based multi-agent systems has expanded to embodied multi-agent systems (EMAS) [3, 13, 17, 23, 25, 35]. Through continuous interaction with their environments, EMASs acquire generalized skills for diverse tasks, advancing the pursuit of artificial general intelligence [21]. Recent efforts increasingly focus on automating agent workflows and architectural design. Dasgupta et al. [7] leverage LLMs for flexible task allocation among embodied agents. Frameworks such as CAMEL [18] and AgentVerse [4] explore scalable, role-based collaboration, while MASTER [10] applies LLM-driven Monte Carlo Tree Search for adaptive agent recruitment and reward estimation. ProAgent [33] emphasizes real-time belief updating and adaptive decision-making, and MaAS [34] introduces an *agentic supernet* to enable query-conditioned architecture optimization. In parallel, Jana et al. [16] propose online scheduling algorithms for hybrid truck–drone delivery.

7. Conclusion

This work presents **DRAMA**, a modular and adaptive framework for dynamic multi-agent coordination. By unifying agents and tasks under a common resource abstraction and introducing an affinity-driven, event-triggered scheduling strategy, DRAMA enables efficient, resilient, and scalable task orchestration in continuously changing environments. Extensive experiments verify its advantages in reducing execution time, mitigating conflicts, and maintaining stable throughput under agent turnover and varying task demands. These results demonstrate DRAMA's potential as a practical framework for real-world multi-agent and embodied intelligence systems, and open promising directions for future research in decentralized, large-scale coordination and resource management.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1

[2] Zhongang Cai, Jianping Jiang, Zhongfei Qing, Xinying Guo, Mingyuan Zhang, Zhengyu Lin, Haiyi Mei, Chen Wei, Ruisi Wang, Wanqi Yin, et al. Digital life project: Autonomous 3d characters with social intelligence. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 582–592, 2024. 1

[3] Jingkai Chen, Jiaoyang Li, Yijiang Huang, Caelan Garrett, Dawei Sun, Chuchu Fan, Andreas Hofmann, Caitlin Mueller, Sven Koenig, and Brian C Williams. Cooperative task and motion planning for multi-arm assembly systems. *arXiv preprint arXiv:2203.02475*, 2022. 8

[4] Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chen Qian, Chi-Min Chan, Yujia Qin, Yaxi Lu, Ruobing Xie, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents. *arXiv preprint arXiv:2308.10848*, 2(4):6, 2023. 5, 8

[5] Yongchao Chen, Jacob Arkin, Yang Zhang, Nicholas Roy, and Chuchu Fan. Scalable multi-robot collaboration with large language models: Centralized or decentralized systems? In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 4311–4317. IEEE, 2024. 1, 8

[6] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240): 1–113, 2023. 1

[7] Ishita Dasgupta, Christine Kaeser-Chen, Kenneth Marino, Arun Ahuja, Sheila Babayan, Felix Hill, and Rob Fergus. Collaborating with language models for embodied reasoning. *arXiv preprint arXiv:2302.00763*, 2023. 8

[8] M Bernardine Dias, Marc Zinck, Robert Zlot, and Anthony Stentz. Robust multirobot coordination in dynamic environments. In *IEEE International Conference on Robotics and Automation, 2004. Proceedings. ICRA'04. 2004*, pages 3435–3442. IEEE, 2004. 1

[9] Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. Incoder: A generative model for code infilling and synthesis. *arXiv preprint arXiv:2204.05999*, 2022. 1

[10] Bingzheng Gan, Yufan Zhao, Tianyi Zhang, Jing Huang, Yusu Li, Shu Xian Teo, Changwang Zhang, and Wei Shi. Master: A multi-agent system with llm specialized mcts. *arXiv preprint arXiv:2501.14304*, 2025. 5, 8

[11] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Yu Wu, YK Li, et al. Deepseek-coder: When the large language model meets programming–the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024. 1

[12] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*, 2024. 1, 8

[13] Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L Griffiths, and Mengdi Wang. Embodied llm agents learn to cooperate in organized teams. *arXiv preprint arXiv:2403.12482*, 2024. 8

[14] Sirui Hong, Xiawu Zheng, Jonathan Chen, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, et al. Metagpt: Meta programming for multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 3(4):6, 2023. 1, 8

[15] Hengyuan Hu and Jakob N Foerster. Simplified action decoder for deep multi-agent reinforcement learning. *arXiv preprint arXiv:1912.02288*, 2019. 1

[16] Saswata Jana, Giuseppe F Italiano, Manas Jyoti Kashyop, Athanasios L Konstantinidis, Evangelos Kosinas, and Partha Sarathi Mandal. Online drone scheduling for last-mile delivery. In *International Colloquium on Structural Information and Communication Complexity*, pages 488–493. Springer, 2024. 8

[17] Zhao Kaiya, Michelangelo Naim, Jovana Kondic, Manuel Cortes, Jiaxin Ge, Shuying Luo, Guangyu Robert Yang, and Andrew Ahn. Lyfe agents: Generative agents for low-cost real-time social interactions. *arXiv preprint arXiv:2310.02172*, 2023. 1, 8

[18] Guohao Li, Hasan Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for" mind" exploration of large language model society. *Advances in Neural Information Processing Systems*, 36: 51991–52008, 2023. 8

[19] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023. 1

[20] Xinyi Li, Sai Wang, Siqi Zeng, Yu Wu, and Yi Yang. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1):9, 2024. 8

[21] Zhuo Li, Weiran Wu, Yunlong Guo, Jian Sun, and Qing-Long Han. Embodied multi-agent systems: A review. *IEEE/CAA Journal of Automatica Sinica*, 12(6):1095–1116, 2025. 8

[22] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024. 1

[23] Xinzhu Liu, Xinghang Li, Di Guo, Sinan Tan, Huaping Liu, and Fuchun Sun. Embodied multi-agent task planning from ambiguous instruction. In *Robotics: Science and Systems*, pages 1–14, 2022. 8

[24] G Ayorkor Mills-Tettey, Anthony Stentz, and M Bernardine Dias. The dynamic hungarian algorithm for the assignment problem with changing costs. *Robotics Institute, Pittsburgh, PA, Tech. Rep. CMU-RI-TR-07-27*, 7, 2007. 4

[25] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023. 1, 8

[26] Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8494–8502, 2018. 2, 5

[27] Xavier Puig, Tianmin Shu, Shuang Li, Zilin Wang, Yuan-Hong Liao, Joshua B Tenenbaum, Sanja Fidler, and Antonio Torralba. Watch-and-help: A challenge for social perception and human-ai collaboration. *arXiv preprint arXiv:2010.09890*, 2020. 5

[28] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, et al. Chatdev: Communicative agents for software development. *arXiv preprint arXiv:2307.07924*, 2023. 1, 8

[29] Yiran Qin, Enshen Zhou, Qichang Liu, Zhenfei Yin, Lu Sheng, Ruimao Zhang, Yu Qiao, and Jing Shao. Mp5: A multi-modal open-ended embodied system in minecraft via active perception. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16307–16316. IEEE, 2024. 1

[30] Tabish Rashid, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020. 1

[31] Alejandro Torreno, Eva Onaindia, Antonín Komenda, and Michal Štolba. Cooperative multi-agent planning: A survey. *ACM Computing Surveys (CSUR)*, 50(6):1–32, 2017. 8

[32] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 1

[33] Ceyao Zhang, Kaijie Yang, Siyi Hu, Zihao Wang, Guanghe Li, Yihang Sun, Cheng Zhang, Zhaowei Zhang, Anji Liu, Song-Chun Zhu, et al. Proagent: building proactive cooperative agents with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 17591–17599, 2024. 5, 8

[34] Guibin Zhang, Luyang Niu, Junfeng Fang, Kun Wang, Lei Bai, and Xiang Wang. Multi-agent architecture search via agentic supernet. *arXiv preprint arXiv:2502.04180*, 2025. 8

[35] Hongxin Zhang, Weihua Du, Jiaming Shan, Qinhong Zhou, Yilun Du, Joshua B Tenenbaum, Tianmin Shu, and Chuang Gan. Building cooperative embodied agents modularly with large language models. *arXiv preprint arXiv:2307.02485*, 2023. 5, 8

[36] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024. 8

[37] Lifeng Zhou and Vijay Kumar. Robust multi-robot active target tracking against sensing and communication attacks. *IEEE Transactions on Robotics*, 39(3):1768–1780, 2023. 1

[38] Qihao Zhu, Daya Guo, Zhihong Shao, Dejian Yang, Peiyi Wang, Runxin Xu, Y Wu, Yukun Li, Huazuo Gao, Shirong Ma, et al. Deepseek-coder-v2: Breaking the barrier of closed-source models in code intelligence. *arXiv preprint arXiv:2406.11931*, 2024. 1